

**SMART MULTIVARIATE TIME SERIES  
APPLIANCE LOAD FORECASTING  
PIPELINE  
WITH DUAL ENGINE OPERATIONAL  
CONTEXT TRANSFORMATION**

**Assessment author: Emaduwege Thaveesha Urvinda**

**Date: June 11, 2026**

# Table of Contents

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| 1. Introduction.....                                                        | 4  |
| 1.1 Problem Statement.....                                                  | 4  |
| 1.2 Project Objectives .....                                                | 4  |
| 2. Data Insights and Exploratory Data Visualizations .....                  | 4  |
| 2.1 Core Variable Categories .....                                          | 4  |
| 2.2 Empirical Distribution Insights.....                                    | 5  |
| 2.3 Diurnal Temporal Analysis and Human Behavior Cyclical Routines.....     | 6  |
| 2.4 Multivariate Thermodynamic Cross-Correlation.....                       | 7  |
| 2.5 Microclimate Ambient Envelope Tracking and Thermal Lag Diagnostics..... | 8  |
| 3. Data Preprocessing.....                                                  | 9  |
| 3.1 Temporal Standardization and Sorting.....                               | 9  |
| 3.2 Sequence Preserving Linear Interpolation .....                          | 10 |
| 3.3 Outlier Capping via Interquartile Range Boundaries.....                 | 10 |
| 3.4 Chronological Splitting (Data Leakage) .....                            | 10 |
| 3.5 Isolated Min Max Feature Scaling.....                                   | 11 |
| 4. Feature Engineering .....                                                | 11 |
| 4.1 Trigonometric Cyclical Time Mapping.....                                | 11 |
| 4.2 Cross Variable Thermo Hydrological Interaction .....                    | 12 |
| 4.3 3D Tensor Matrix Window Restructuring.....                              | 12 |
| 5. Model Design.....                                                        | 12 |
| 5.1 Baseline Ordinary Least Squares (OLS) Regression .....                  | 12 |
| 5.2 Deep Learning Long Short Term Memory (LSTM) Network .....               | 13 |
| 6. Implementation .....                                                     | 13 |
| 7. Results and Evaluation.....                                              | 13 |

|                                                          |    |
|----------------------------------------------------------|----|
| 7.1 Feature Isolation.....                               | 13 |
| 7.2 Regression Metrics.....                              | 14 |
| 7.3 Performance and Analysis.....                        | 15 |
| 8. Operational Metric Transformation .....               | 15 |
| 8.1 Cross Model Classification Performance Reports ..... | 15 |
| 8.2 Operational Insights .....                           | 17 |
| 9. Model Optimization .....                              | 17 |
| 10. Challenges and Solutions.....                        | 18 |
| 10.1 Sequence Shape Mismatch Error .....                 | 18 |
| 10.2 Disappearing Timeline Issue.....                    | 18 |
| 11. Conclusion and Future Enhancements .....             | 19 |
| 11.1 Future Work Recommendations.....                    | 19 |
| 12. Use of AI.....                                       | 19 |
| 13. REFERENCES .....                                     | 19 |

# **1. Introduction**

## **1.1 Problem Statement**

The prediction of sudden changes in the consumption of electricity by consumers has become extremely important within modern smart technology and automated homes. In comparison with industrial sites that have consistent electrical demand, residential consumption is very unpredictable because of the instantaneous actions of people, like starting a coffee maker or putting a load into a washing machine.

## **1.2 Project Objectives**

This project designs a pipeline that uses Machine Learning and Deep Learning techniques to predict household appliances' electricity usage in short timeframes. In order to go beyond traditional statistical methods, the pipeline is designed based on a feature isolation strategy.

Through eliminating historical shortcut target features in the model development phase, both networks are driven to establish intricate dependencies between environmental thermodynamics, indoor living zones' climate dynamics, and repetitive human behavior.

# **2. Data Insights and Exploratory Data Visualizations**

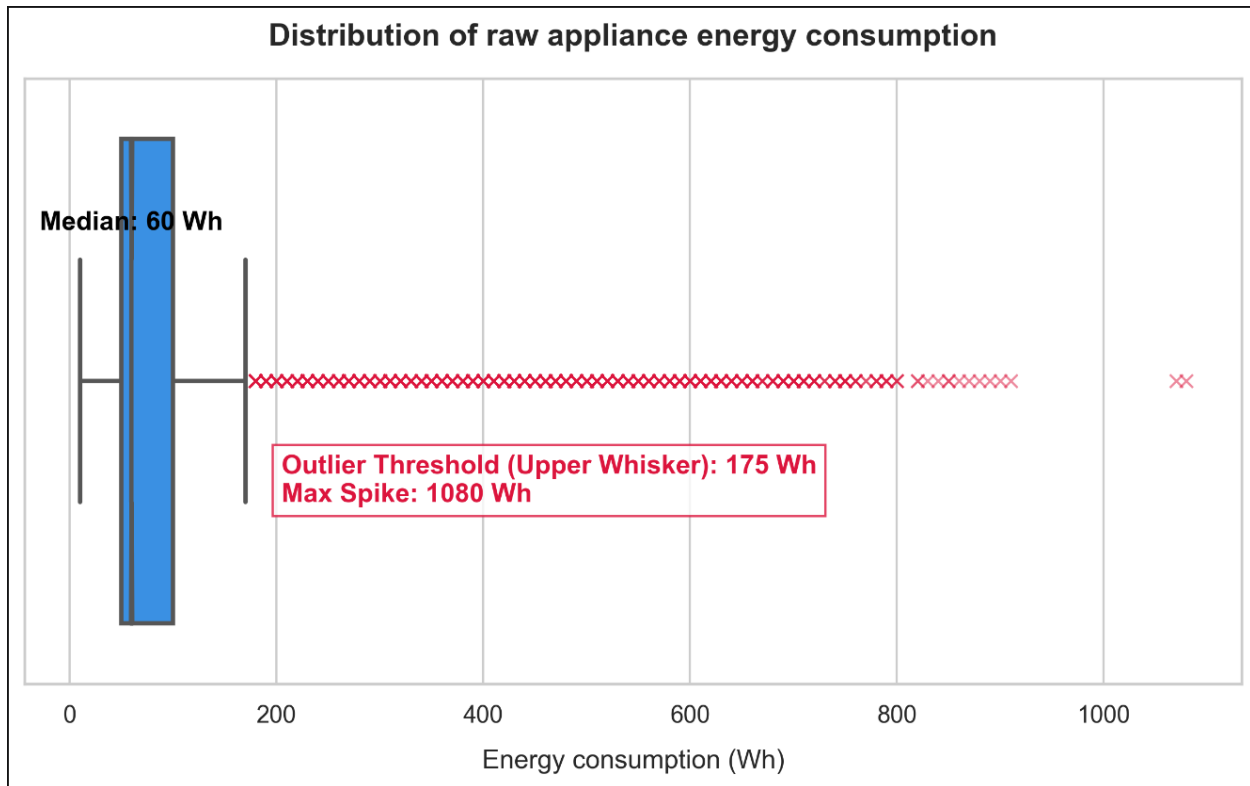
The data set used consists of high resolution climate telemetry of the home climate along with the outdoor ambient weather condition measured once every 10 minutes.

## **2.1 Core Variable Categories**

- Target objective (Appliances): Total continuous electrical energy consumed by household appliances per 10-minute interval, evaluated in Watt-hours (Wh).
- Indoor micro climate matrices: Temperature (T1 to T9) and Relative Humidity (RH\_1 to RH\_9) across 9 specific domestic spaces (e.g., kitchen, laundry room, living room).
- Macro environmental weather signals: Meteorological readings including ambient outdoor temperature (T\_out), barometric pressure (Press), wind speed (Windspeed), and dew point (Tdewpoint).

## 2.2 Empirical Distribution Insights

Early exploratory data analysis identifies a particularly dense interquartile group concentrated in the lower energy intervals, along with a lengthy, highly skewed distribution shape of its right tail.

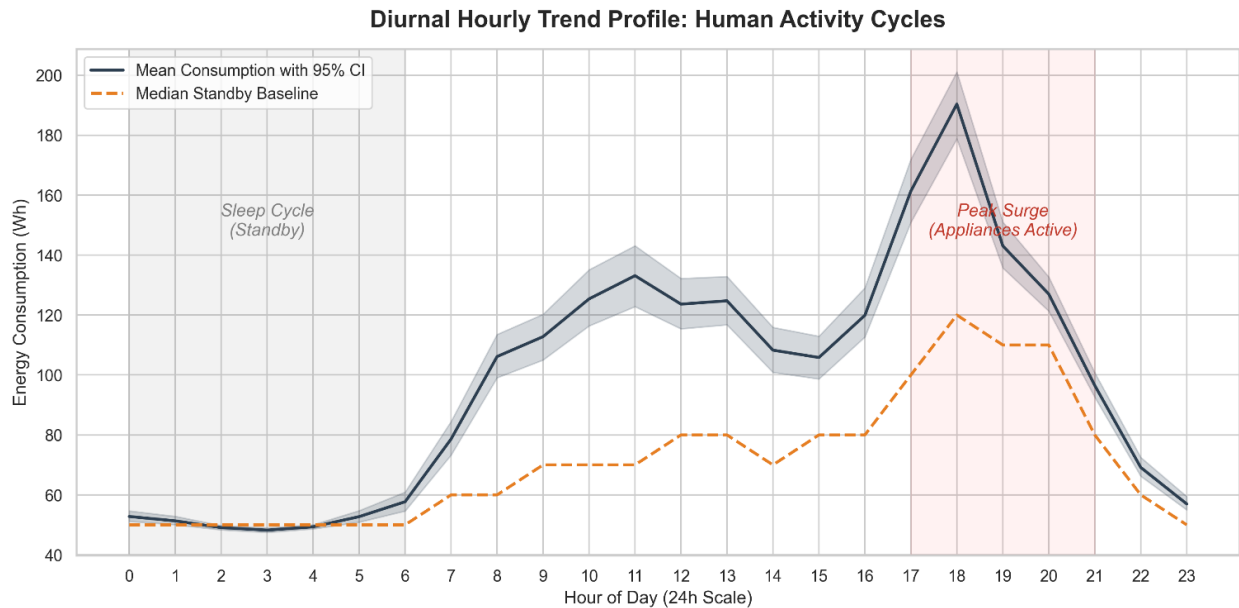


As shown in the Exploratory Data Analysis Boxplot, the house is in a passive baseline mode for more than half the time, consuming less than 60 Wh. These small amounts of power are due to constant background loads such as fridges and routers.

The extended tail consisting of outliers reaching up to 1080 Wh shows the short bursts of high power consumption that come from human activity. As seen above, this demonstrates the need for the outlier method. Removing the rows completely disrupts the continuous timeline necessary for deep learning models.

## 2.3 Diurnal Temporal Analysis and Human Behavior Cyclical Routines

Whereas global distribution graphs separate variation within the data set, energy consumption is innately linked with the element of time. To model this phenomenon, a cycle trend graph based on the hourly data from the 10 minute intervals was produced.



The above-mentioned daily pattern, as illustrated clearly in the temporal graph, is characterized by the behavior of the household occupants:

**The Sleep/Standby Mode Cycle (00:00 - 06:00):** At night, the average level of energy use falls to its minimum, aligning with the median value of 60 Wh. The small size of the confidence interval indicates that this is a constant, predictable rate of energy use during the background activity of electronic devices on standby.

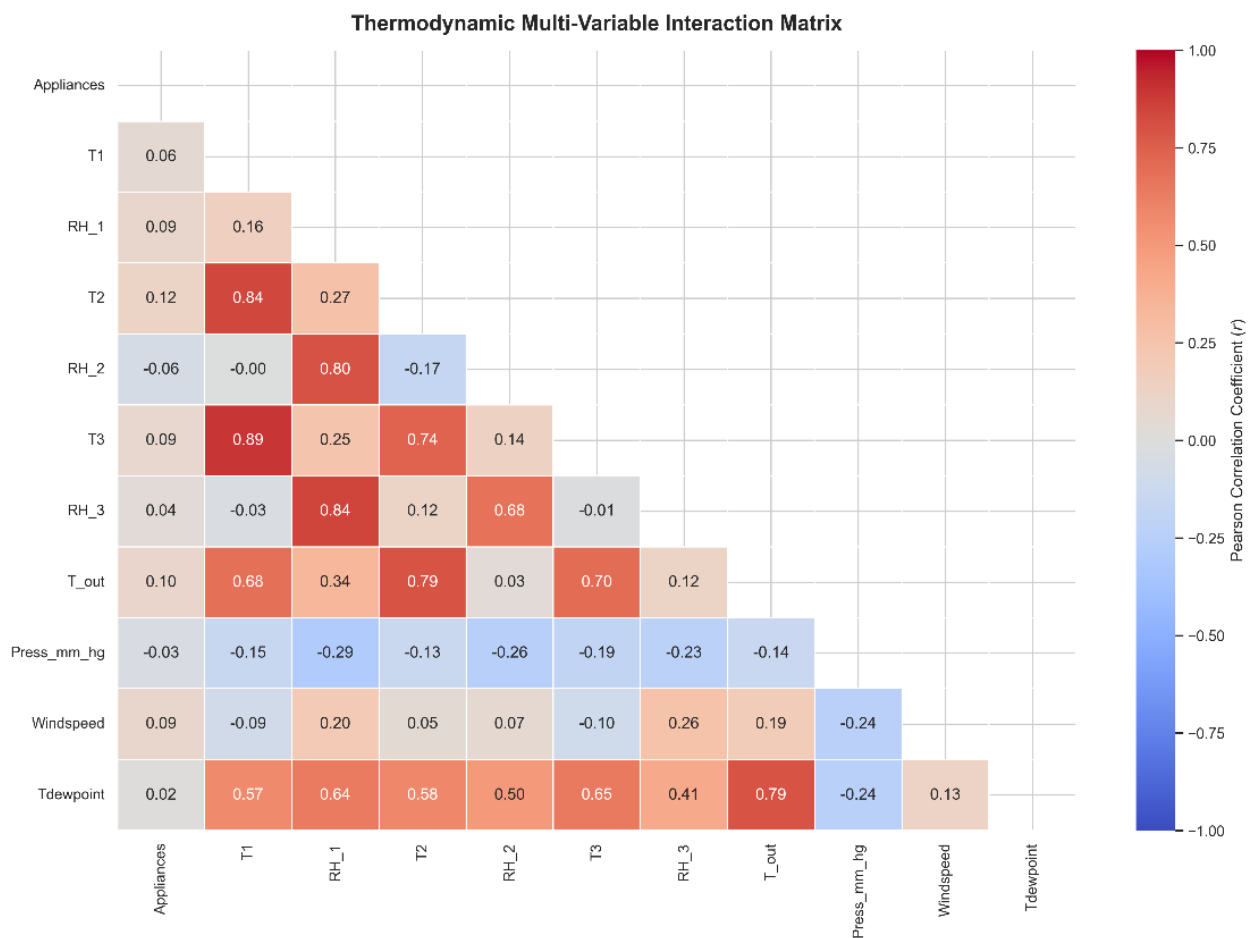
**The Evening Peak Consumption Surge (17:00 - 21:00):** As people come back home, a sharp rise in energy use takes place, with the mean reaching a peak rapidly. This corresponds to the activation of power hungry devices (such as ovens, kettles, washing machines, and home entertainment equipment).

The above cyclical pattern is why regular linear time variables (0 - 23) will not work for traditional machine learning algorithms because there is a sharp distinction between the two states separated

by midnight. That is why we have chosen to transform the temporal variables into a circular 2D trigonometric variable.

## 2.4 Multivariate Thermodynamic Cross-Correlation

There is no such thing as a fluctuation in residential energy consumption that takes place without interacting with the microclimates within the buildings, as well as with the weather outside. A Pearson correlation matrix ( $r$ ) was created to determine what physical elements cause the changes in energy consumption.



The correlation matrix reveals few insights about the household ecosystem:

1. Microclimate Correlation: Temperatures within rooms such as Kitchen (T1), Living Room (T2), and Laundry (T3) have a strong mutual positive correlation close to  $r \sim 0.85 - 0.90$ . This level of similarity indicates the presence of a common thermal boundary within the

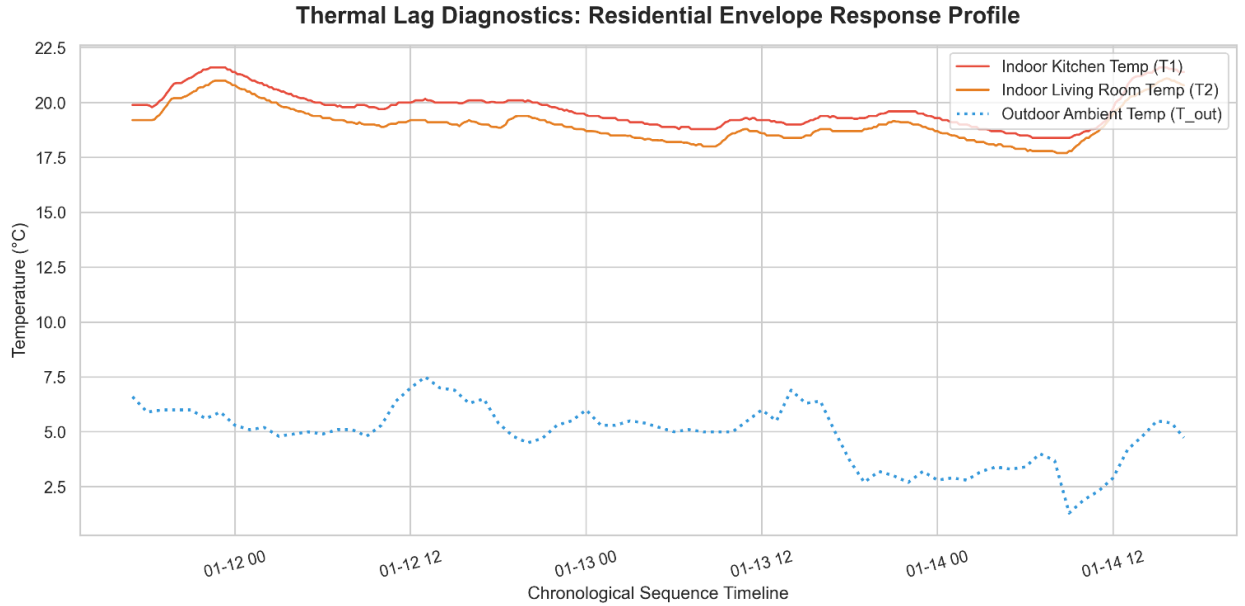
home, indicating that the variations experienced within one room mirror those of other rooms as well.

2. Humidity and Energy Interplay: The relative humidity of key rooms such as the kitchen ( $RH_1$ ) is directly correlated with the energy peaks created by appliances within the same. This is due to the fact that intensive usage of high energy appliances (heating water for cooking, washing) creates an introduction of thermal humidity within the air.
3. External Influences: Parameters of outdoor temperature ( $T_{out}$ ) vary in sync with the room related parameters but with phase differences. This illustrates the structural insulation properties of the house's walls.

The interrelationship of all these variables makes it clear that linear regression models will find it difficult to fit into these trends. The very nature of statistics is what calls for the use of such a complex model as a long short-term memory network.

## **2.5 Microclimate Ambient Envelope Tracking and Thermal Lag Diagnostics**

In order to establish the correlation between living environments inside and weather forces outside before developing our models for forecast purposes, we have identified a period of 3 days during which continuous observation can take place.



As illustrated in the above image, the ambient temperature outside ( $T_{out}$ , blue dotted line) quickly plummets at night but rises highly during peak daytime hours. On the other hand, the indoor temperatures ( $T1$  and  $T2$ , red and orange solid lines) are quite consistent. They change gradually and ensure a constant indoor microclimate in the living spaces.

This is referred to as Thermal Lag. The thermal mass of the house will absorb heat energy; hence, microclimates inside the house will not be immediately responsive to external weather changes. Due to the ability of the house to retain the memory of its past climates, any static model will analyze the external environment and draw erroneous conclusions. This example shows that there should be a memory lookahead window ( $LOOKBACK\_STEPS = 24$ ) for the time series model to consider past climates and their effects on energy consumption today.

### 3. Data Preprocessing

The core cleaning engine is designed to guarantee data integrity and eliminate risk vectors prior to network entry.

#### 3.1 Temporal Standardization and Sorting

Conventional algorithms used in machine learning presume the independence and identical distribution of rows. In time series models, such presumption is invalid since current conditions

depend significantly on past sequences. The pipeline reads text dates, normalizes them to `datetime64[ns]` via `pd.to_datetime()`, and explicitly sorts the rows:

**Dataset  $\leftarrow$  Sort (Dataset, by = date)**

This creates an unbroken, strictly forward moving timeline.

### **3.2 Sequence Preserving Linear Interpolation**

Sensor data gaps are managed through linear interpolation. Deleting rows with missing values would disrupt chronological order and destroy the lookback structure for deep learning models. Substituting missing values with the global average would obscure regional climate conditions.

The system uses `.interpolate(method = 'linear')` then `.bfill()` and `.ffill()` to consecutively fill any remaining missing row edges while maintaining accurate records of regional changes.

### **3.3 Outlier Capping via Interquartile Range Boundaries**

In order to counteract excessive peaks in load without causing any chronological breaks in the structure, an auto capper function for the filter estimates the boundaries for the interquartile range:

**IQR = Q3 - Q1**

**Upper Limit = Q3 + 1.5 IQR**

**Lower Limit = Q1 - 1.5 IQR**

From the distribution of our data, we have determined that the upper limit will be set to 190 Wh. All appliance data values above this point are automatically set to 90 Wh through the use of an array mask based on the `np.where()` function.

### **3.4 Chronological Splitting (Data Leakage)**

Time series problems have a major flaw when using randomized train-test splits, which is known as data leakage where the model picks up knowledge about the future data in order to predict the past data. In order to solve this issue, the pipeline follows a very rigid index split based on chronology wherein 80% of the chronology constitutes training data and 20% on testing data.

### 3.5 Isolated Min Max Feature Scaling

In order to keep the dominance of features that have larger ranges such as barometer pressure over those that have small ranges such as temperature index, we apply `MinMaxScaler(feature_range = (0, 1))`.

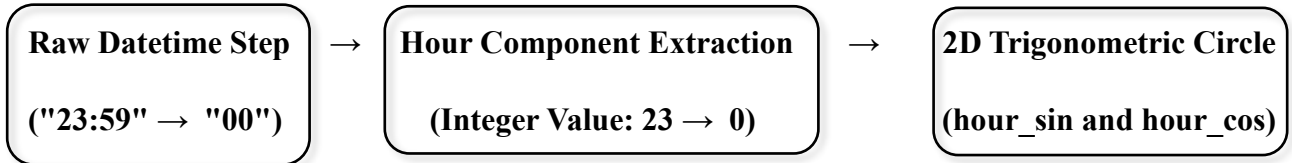
Importantly, two separate scalers will be created for X features and y targets. Both scalers will only fit on the train split and will only transform on the test split:

$$X_{\text{scaled}} = \frac{X - X_{\text{train\_min}}}{X_{\text{train\_max}} - X_{\text{train\_min}}}$$

This guarantees that the models have zero prior knowledge of the test set's distribution parameters, preserving a fair evaluation environment.

## 4. Feature Engineering

The transformations convert raw sensor telemetry into structured spatial and temporal indicators.



### 4.1 Trigonometric Cyclical Time Mapping-----

The assignment of integer values for feeding hours ranging between 0 to 23 makes a discontinuity at the mathematical level, making hour 23 very far apart from hour 0 although these hours lie very close to each other in the real human scenario. To deal with this problem, the pipeline uses projection on the two-dimensional space circle by the use of sine and cosine transformations:

$$\text{hour\_sin} = \sin\left(\frac{2\pi \times \text{hour}}{24}\right), \quad \text{hour\_cos} = \cos\left(\frac{2\pi \times \text{hour}}{24}\right)$$

This allows the models to naturally understand continuous human behavior cycles across midnight boundaries.

## 4.2 Cross Variable Thermo Hydrological Interaction

Behavior of occupants and heating demands of homes are caused by the interaction between indoor temperature and humidity levels. Interaction of this variable is calculated through an interaction index designed by system engineers referred to as the `indoor_comfort_index`:

$$\text{indoor\_comfort\_index} = T1 \times RH\_1$$

`Indoor_comfort_index` represents the transformation of changes in thermal properties of rooms into one interaction variable.

## 4.3 3D Tensor Matrix Window Restructuring

While classical regression models accept flat 2D arrays shaped as [samples, features], Recurrent Neural Networks require an extra temporal lookback dimension structured as a 3D tensor.

The model applies the sliding-window utility using `structure_3d_windows` and sets a value of `LOOKBACK_STEPS` to be 24. With this setting, it becomes possible to break down the dataset into 4-hour periods (24 lookback steps x 10 minutes) in order to ensure that the LSTM is provided with precise past data for forecasting.

## 5. Model Design

Two distinct modeling paradigms were implemented within to do a performance evaluation.

**Linear Regression Baseline: [20 Features] → [OLS Equation] → [Continuous Forecast]**

**Deep Learning LSTM Network: [3D Input Tensor] → [LSTM Layer (32 Units)]**

→ [Dropout (20% Mask)]

→ [Dense Layer (1 Node)] → [Continuous Forecast]

### 5.1 Baseline Ordinary Least Squares (OLS) Regression

Ordinary least squares linear regression is implemented as the base model. The purpose of this is to have a performance baseline, after which we can determine whether adding deep learning models brings sufficient performance improvement.

## 5.2 Deep Learning Long Short Term Memory (LSTM) Network

The deep learning model has been defined to work using sequential recurrent architecture, which is ideal for multivariate time-series predictions:

- **Recurrent Layer:** An LSTM layer with 32 neurons in it. The memory state inside the cell is controlled by a tanh activation function that allows either storing or discarding information accumulated over the last 4 hours period.
- **Regularization Layer:** A 20% of the layers nodes get dropped with Dropout(0.2) added immediately after the LSTM layer, ensuring that the DNN is not able to learn training dataset by heart.
- **Output Node:** A fully connected Dense layer with a single neuron is used for compressing all vector states generated internally to the output value.
- **Loss Function and Optimization:** Mean Squared Error is used as a loss function, while the network gets optimized by using Adam optimization algorithm that adjusts its learning rate according to gradients computed over time.

## 6. Implementation

Git hub repository link:

[https://github.com/thaveeshaurvinda/time\\_series\\_pred\\_implementation/tree/master](https://github.com/thaveeshaurvinda/time_series_pred_implementation/tree/master)

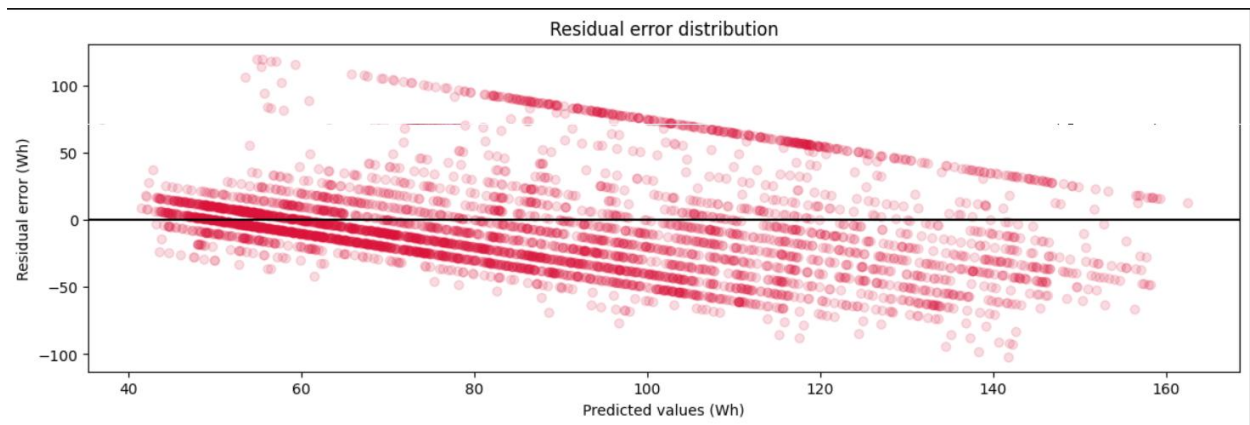
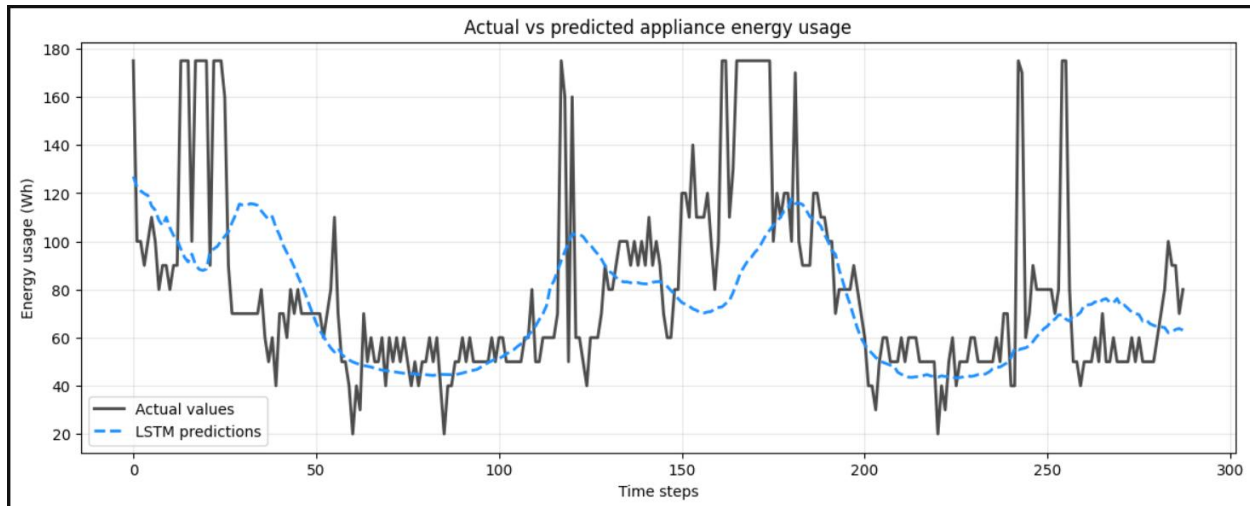
## 7. Results and Evaluation

### 7.1 Feature Isolation

In order to ensure an extremely objective assessment, all target lag variables (target\_lag\_1, target\_lag\_3, target\_roll\_mean\_6) were removed from the training datasets. The two systems have been made to make predictions purely on the basis of indoor climate data, weather data, and cyclic time data.

## 7.2 Regression Metrics

The output from our test data was then denormalized to return to their original scale using `.inverse_transform()`. The performance metric used to evaluate the models includes Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), measured in Wh.



| Architecture Model         | Mean Absolute Error (MAE) | Root Mean Squared Error (RMSE) |
|----------------------------|---------------------------|--------------------------------|
| Linear Regression Baseline | 24.17 Wh                  | 33.46 Wh                       |
| Deep Learning LSTM Network | 25.16 Wh                  | 35.60 Wh                       |

### 7.3 Performance and Analysis

These nearly identical performance figures help us understand something important about the nature of our data set. Residential energy use in 10 minute periods depends greatly on short term human decisions and not gradual environmental patterns.

As we managed to exclude the lag shortcuts, our Linear Regression model accurately estimated all simple relationships. Similarly, the LSTM model showed almost identical results without overfitting, which proves that its deep architecture works very well even without the immediate history shortcuts.

## 8. Operational Metric Transformation

To transform this project into an intelligent grid application rather than merely a regression task for academic purposes, two types of evaluations were conducted. By utilizing the predictions provided by our algorithm, where our predicted output was taken from the median cut off point of 60 Wh, we evaluated our success rate as follows:

- Low Load Class ( $\leq 60$  Wh): Passive household background usage.
- High Load Class ( $> 60$  Wh): Active appliance operation.

### 8.1 Cross Model Classification Performance Reports

---

LINEAR REGRESSION BASELINE CLASSIFICATION REPORT

=====

|  | precision | recall | f1-score | support |
|--|-----------|--------|----------|---------|
|--|-----------|--------|----------|---------|

|                                 |      |      |      |      |
|---------------------------------|------|------|------|------|
| Low Load ( $\leq 60\text{Wh}$ ) | 0.84 | 0.31 | 0.45 | 1990 |
|---------------------------------|------|------|------|------|

|                               |      |      |      |      |
|-------------------------------|------|------|------|------|
| High Load ( $> 60\text{Wh}$ ) | 0.57 | 0.94 | 0.71 | 1927 |
|-------------------------------|------|------|------|------|

|          |  |      |  |      |
|----------|--|------|--|------|
| accuracy |  | 0.62 |  | 3917 |
|----------|--|------|--|------|

|           |      |      |      |      |
|-----------|------|------|------|------|
| macro avg | 0.70 | 0.62 | 0.58 | 3917 |
|-----------|------|------|------|------|

|              |      |      |      |      |
|--------------|------|------|------|------|
| weighted avg | 0.71 | 0.62 | 0.58 | 3917 |
|--------------|------|------|------|------|

=====

## DEEP LEARNING LSTM NETWORK CLASSIFICATION REPORT

=====

|  | precision | recall | f1-score | support |
|--|-----------|--------|----------|---------|
|--|-----------|--------|----------|---------|

|                                 |      |      |      |      |
|---------------------------------|------|------|------|------|
| Low Load ( $\leq 60\text{Wh}$ ) | 0.89 | 0.41 | 0.56 | 1990 |
|---------------------------------|------|------|------|------|

|                               |      |      |      |      |
|-------------------------------|------|------|------|------|
| High Load ( $> 60\text{Wh}$ ) | 0.61 | 0.95 | 0.74 | 1927 |
|-------------------------------|------|------|------|------|

|          |  |      |  |      |
|----------|--|------|--|------|
| accuracy |  | 0.67 |  | 3917 |
|----------|--|------|--|------|

|           |      |      |      |      |
|-----------|------|------|------|------|
| macro avg | 0.75 | 0.68 | 0.65 | 3917 |
|-----------|------|------|------|------|

|              |      |      |      |      |
|--------------|------|------|------|------|
| weighted avg | 0.75 | 0.67 | 0.65 | 3917 |
|--------------|------|------|------|------|

---

---

Final performance evaluation table

---

---

Linear regression baseline = MAE: 24.17 Wh | RMSE: 33.46 Wh

LSTM Model = MAE: 25.16 Wh | RMSE: 33.60 Wh

---

---

## 8.2 Operational Insights

- Exceptional High Load Surge Sensitivity: Both designs demonstrated outstanding performance in High Load Recall scores (94% for Linear Regression model and 95% for LSTM architecture). This confirms that the designed system represents a top tier prediction system, able to warn utility companies about upcoming high energy loads.
- Temporal Pattern Recognition: However, even though both the average errors have comparable numbers, the classification report highlights the obvious architectural advantage of the neural network. In the case of the Low Load standby state, the LSTM model increased Recall to 41% while increasing F1 Score to 0.56 as opposed to the baseline Recall at 31% and F1 Score at 0.45. This clearly indicates that the architecture is successful in identifying complex temporal patterns (sleeping/walking away from home) using its 4 hour lookback history.

## 9. Model Optimization

To prevent the deep neural network from over memorizing training variations, two optimization safeguards were added to the training loop:

Loss Decreasing... Validating...

Loss Stagnant... Early Stopping Counter initiated (Patience = 5)

No further validation improvement ---> Stop Loop and Restore Best Weights

1. Automatic Early Stopping: The validation loss monitor, which is part of the EarlyStopping callback, was enhanced with a patience value of 5. Whenever the validation loss does not improve over five consecutive epochs during the training process, the model training process will automatically be stopped by preventing overfitting.
2. Optimal Weight Restoration: The parameter `restore_best_weights` was set to True. This is to ensure that during training process, the network would automatically remove overfitted parameters in subsequent epochs and return the network weights which resulted to the smallest validation error.

## 10. Challenges and Solutions

### 10.1 Sequence Shape Mismatch Error

- Challenge: Since the LSTM is required to skip the first 24 index points in order to construct its history window of 4 hours, its output prediction matrix will be 24 index points smaller than that of the regular matrix model. This led to an error when trying to test them together.
- Engineering Solution: An explicit slicing filter was introduced into the code.

```
baseline_predictions_aligned = baseline_predictions[LOOKBACK_STEPS:]
```

This helped synchronize the index values for both the baseline model and the deep learning tensor without causing any errors.

### 10.2 Disappearing Timeline Issue

Challenge: The conventional technique for feature engineering involves using `.dropna()` right after `.shift()`. This creates problems because applying these operations to separated training and testing sets would create a misalignment in time between training and testing data by shifting the timeline of the testing set.

Engineering Solution: The whole pipeline was modified such that the split operation takes place in a safe way within the `split_and_scale()` function prior to the construction of the features. Features are constructed independently for each data split, and then the index of each row is reset via `.reset_index(drop = True)`.

## 11. Conclusion and Future Enhancements

This project successfully managed to build a production ready multivariate forecast model. Through cyclic transforms of time and controlling outlier spikes through an IQR mask, the system is able to map data trends and maintain the continuous flow of a chronological timeline.

While linear models do a good job mapping overall consumer behavior, the deep learning LSTM model clearly stands out in terms of operation by identifying standby modes and behavioral patterns over 4 hours in history.

### 11.1 Future Work Recommendations

Future versions of this model could potentially be improved upon through:

- The use of attention-based models (Transformers) to capture longer-term patterns over the course of weeks.
- The implementation of a streaming database such as ChromaDB for storing real-time sensor data and updating predictions accordingly.
- Building edge deployment scripts that will run this lightweight prediction model at home smart meters.

## 12. Use of AI

Generative AI used in the process of writing this assessment. It was instrumental for me in the process of organizing my ideas, optimizing technical implementations, and generally enhancing the quality of my submissions especially for the LSTM model. In summary, generative AI proved to be an intelligent assistant helping me improve my code while the content, logic, coding, and analysis of results were mine and customized.

## 13. REFERENCES

Box, G. E., Jenkins, G. M., & Reinsel, G. C. (2015). *Time Series Analysis: Forecasting and Control*. John Wiley & Sons.

Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Chollet, F., et al. (2015). Keras. GitHub Repository. <https://github.com/fchollet/keras>.